Formatos de archivo: CSV, JSON, XML

¿Cómo viajan los datos en el mundo real?

Programación Científica y Técnica — 2026

Alfonso Pintos - Guadalupe Artigas

Made with GAMMA

Antes de empezar...

> "Descargaste un dataset de datos abiertos de Uruguay. Lo abrís y ves esto:"

```
nombre;edad;depto
"María, López";23;Montevideo
Juan Pérez;NULL;
```

Pregunta:

¿Qué problemas ven?

Observá el fragmento de datos y listá todo lo que te parece incorrecto o ambiguo.

¿Cuántos problemas pueden identificar en 30 segundos?

Ejemplo: Comillas con coma dentro, NULL como texto

Made with GAMMA

¿Por qué importa esto?

El 80% del trabajo con datos es… limpiar datos

- En la vida real, los datos **nunca llegan limpios**.
- El formato del archivo define **cómo se leen, se rompen y se reparan**.
- Si no entendés el formato, la IA tampoco te va a salvar: **garbage in, garbage out**.
- Hoy: los **3 formatos más comunes** en ciencia de datos y APIs.

Made with GAMMA

¿Qué vamos a ver hoy?

CSV

El más simple

JSON

El favorito de las APIs.

XML

El veterano que sigue vivo.

Comparación práctica

Cuándo usar cada uno.

Actividad

"Rescatar" un dataset roto.

Made with GAMMA

CSV — Simple, universal, peligroso

- Texto plano, una fila por línea, campos separados por un delimitador.
- **No tiene estándar único** (RFC 4180 existe, pero pocos lo respetan).
- Cada fila = un registro. Primera fila = header (generalmente).

Ejemplo "feliz":

```
nombre,edad,ciudad
Ana,21,Montevideo
Pedro,22,Salto
José,20,Artigas
```

Made with GAMMA

CSV — Lo que puede salir mal (y sale)

Delimitador

¿Coma, punto y coma, tab? Depende del país/sistema. En Uruguay muchos archivos usan `;` porque la coma se usa en decimales.

Encoding

UTF-8 vs Latin-1 vs Windows-1252. Los acentos (á, é, ñ) se rompen si no coincide.

Comillas

Si un campo contiene el delimitador (`"López, María"`), debe ir entre comillas. ¿Y si tiene comillas adentro?

Faltantes

¿Campo vacío, "NULL", "N/A", "-", espacio? Cada uno se interpreta distinto.

Tipos

Todo es texto. ¿"23" es número o string? ¿"01/03/2026" es 1 de marzo o 3 de enero?

Mensaje clave: *CSV parece fácil, pero la mayoría de los bugs en pipelines de datos empiezan acá.*

Made with GAMMA

Leer y escribir CSV en Python

Con la librería estándar:

```
import csv

with open("datos.csv", encoding="utf-8") as f:
    reader = csv.DictReader(f, delimiter=";")
    for row in reader:
        print(row)  # dict por fila
```

Decisiones que importan (y que la IA no toma por vos):

¿Qué encoding uso?

→ Depende del archivo real.

¿Qué delimiter?

→ Hay que inspeccionar o preguntar.

¿Qué hago con faltantes?

→ Decisión de negocio/análisis.

Made with GAMMA

Ejercicio relámpago

Se les da un CSV con estos problemas:

Delimitador `;`	Campo con coma dentro del texto
Valor `"N/A"` en edad	

Pregunta:

- ¿Qué pasa si uso `delimiter=","` por defecto?
- ¿Cómo detecto que `"N/A"` no es un número antes de que explote?

Made with GAMMA

SEMANA 2

JSON — El idioma de las APIs

- Formato de texto estructurado: pares **clave-valor**.
- Soporta **tipos nativos**: string, number, boolean, null, array, object.
- **Anidamiento**: un valor puede ser otro objeto o una lista.
- Es el formato estándar de casi toda API web moderna.

Ejemplo:

```
{
 "nombre": "Ana",
 "edad": 21,
 "materias": [
 "Programación",
 "Álgebra"
 ],
 "activa": true
}
```

Made with GAMMA

JSON — Lo bueno y lo complicado

Ventajas

- Tipos explícitos (`21` es número, `"21"` es texto, `null` es null).
- Anidamiento natural (listas dentro de objetos, objetos dentro de listas).
- Legible por humanos y por máquinas.

Trampas reales

- **Anidamiento profundo**: ¿cómo "aplanás" 5 niveles para hacer una tabla?
- **Inconsistencia entre registros**: un registro tiene campo "mail", otro tiene "email".
- **Archivos grandes**: JSON puro no se lee bien línea por línea (existe JSON Lines / NDJSON para eso).
- **Sin comentarios**: no podés documentar dentro del archivo.

Made with GAMMA

Leer y escribir JSON en Python

```
import json
# Leer
with open("datos.json", encoding="utf-8") as f:
    data = json.load(f)  # lista de dicts o dict
# Escribir
with open("salida.json", "w", encoding="utf-8") as f:
    json.dump(data, f, ensure_ascii=False, indent=2)
```

Decisiones que importan:

¿ensure_ascii=False? para que los acentos queden bien (ñ, á, etc.).

¿Claves inconsistentes? normalización entre registros.

¿Anidamiento? decidir si aplano o mantengo estructura.

Made with GAMMA

XML — El veterano que sigue vivo

Key points:

- Formato de etiquetas (tags) anidadas, similar a HTML.
- Muy usado en: gobierno, banca, facturación electrónica, sistemas legacy.
- En Uruguay: la facturación electrónica (CFE/DGI) usa XML.

Ejemplo:

```
<estudiantes>
  <estudiante>
    <nombre>Ana</nombre>
    <edad>21</edad>
    <ciudad>Montevideo</ciudad>
  </estudiante>
</estudiantes>
```

Made with GAMMA

XML — Lo bueno y lo pesado

Ventajas

- Estructura explícita y validable (con esquemas XSD).
- Atributos + elementos → más expresivo.
- Estándar en muchos sistemas institucionales.

Trampas reales

- Verboso: mucho texto para poca información (comparar con JSON equivalente).
- Parsing más complejo que CSV o JSON.
- Namespaces: pueden complicar el acceso a elementos.
- Múltiples formas de representar lo mismo: ¿atributo o sub-elemento?

Made with GAMMA

Leer XML en Python

Utilizando la librería estándar `xml.etree.ElementTree`:

```
import xml.etree.ElementTree as ET
tree = ET.parse("datos.xml")
root = tree.getroot()
for est in root.findall("estudiante"):
    nombre = est.find("nombre").text
    edad = est.find("edad").text  # siempre string
    print(nombre, edad)
```

Decisiones que importan:

Todo viene como texto: hay que convertir tipos explícitamente.

¿Datos en atributos o sub-elementos?: hay que inspeccionar.

¿Namespaces?: si hay, cambia la forma de buscar.

Made with GAMMA

CSV vs JSON vs XML — ¿Cuándo uso cada uno?

Aspecto	CSV	JSON	XML
Estructura	Plana (tabla)	Anidada (árbol)	Anidada (árbol)
Tipos	Todo es texto	Nativos (num, bool, null)	Todo es texto
Legibilidad	Alta (si es simple)	Alta	Media-baja
Tamaño	Compacto	Medio	Verboso
Uso típico	Datasets, exports	APIs, configs	Gobierno, banca, legacy
Anidamiento	No soporta	Natural	Natural
Validación formal	No tiene	JSON Schema (opcional)	XSD (maduro)
En Uruguay	Datos abiertos, INE	APIs modernas	Facturación electrónica DGI

No hay formato "mejor". Hay formato adecuado al contexto. Un profesional de datos sabe trabajar con los tres.

Made with GAMMA

La tarea más común: "me llega en X, necesito en Y"

Conversiones típicas:

- CSV → JSON (para enviar a una API).
- JSON → CSV (para abrir en Excel o cargar en base).
- XML → JSON o CSV (para procesar datos de gobierno/banca).

Pregunta:

Si tengo un JSON anidado (listas dentro de objetos), ¿cómo lo convierto a CSV que es plano?

¿Qué información puedo perder?

Aplanar = decisión de diseño, no mecánica.

Made with GAMMA

Encoding — Por qué "se rompen los acentos"

Encodings comunes

UTF-8

Estándar moderno, soporta todo (ñ, á, emojis, kanji).

Latin-1 / ISO-8859-1

Común en archivos viejos de Windows en español.

Windows-1252

Parecido a Latin-1, con diferencias sutiles.

Síntomas de encoding incorrecto:

"InformaciÃ³n" en vez de "Información"

"AÃ±o" en vez de "Año"

Regla práctica:

```
open("archivo.csv", encoding="utf-8")
```

Siempre especificá encoding al abrir archivos. Nunca confíes en el default del sistema.

Made with GAMMA

Valores faltantes: un problema de decisión, no de código

Un mismo "vacío" puede aparecer como:

```
"", "N/A", "null", "NULL", "-", " ", "None", ".", "n/d", "s/d"
```

Pregunta crítica:

- ¿"vacío" significa "no se sabe", "no aplica", o "se olvidaron de llenar"?
- ¿Descarto el registro, imputo un valor, o lo marco?
- **Esta decisión cambia el resultado del análisis.**

Mensaje:

> ***La IA puede escribir el código para reemplazar faltantes. Pero vos decidís qué significa ese faltante y qué hacer con él. Eso es criterio profesional.***

Made with GAMMA

Actividad: Rescatá el dataset

Verán en webasignatura **un mini-dataset "roto"** con estos problemas intencionados:

1. Un CSV con delimitador `;`, encoding `latin-1`, y campos con comas internas.
2. Un JSON con claves inconsistentes (`nombre` vs `name`, `mail` vs `email`).
3. Valores faltantes en distintas formas (`""`, `"N/A"`, `null`).

Tareas (en parejas):

- Cargar ambos archivos correctamente.
- Unificar a un esquema común.
- Documentar: qué problemas encontraron y qué decidieron hacer.
- Escribir 2 checks de calidad simples.

Entrega: notebook o script + comentarios con decisiones.

Made with GAMMA

Proyecto 1

Lo que practicamos hoy

Lo que practicamos hoy es lo que van a hacer en el Proyecto 1: Mini-ETL.

- Próxima semana:
  - Pipeline de limpieza + funciones + checks de calidad.
  - Regex para limpiar campos de texto.

Acción para la semana:

- Elegir dataset/fuente para el proyecto (pueden consultar datos abiertos de Uruguay, Kaggle, o fuente propia).
- Crear el repo con la estructura base y el README inicial.

Made with GAMMA